CLOUD COMPUTING

Session 6 :

AWS Well Architected Framework:

- Reliability Pillar
- Performance Efficiency
- Cost Optimization

APROBAT

Conf. univ. dr. ing. IUSTIN PRIESCU - UTM
Dr. ing. Sebastian NICOLAESCU - Verizone Bussines-US

AWS Well Architected Framework

The goal of this framework is to enable customers to:

- Assess and improve their architectures
- Better understand the business impact of their design decisions

It provides a set of questions developed by AWS experts to helps customers think critically about their architecture.

It asks, "Does your infrastructure follow best practices?"

AWS Well Architected Framework

Architects should leverage the AWS Well-Architected Framework in order to:

- Increase awareness of **architectural best practices**.
- Address **foundational areas** that are often neglected.
- **Evaluate** architectures using a consistent set of principles.

AWS Well Architected Framework

The AWS Well-Architected Framework does not provide:

- Implementation details
- Architectural patterns
- Relevant case studies

However, it does provide:

- Questions centered on critically understanding architectural decisions
- Services and solutions relevant to each question
- References to relevant resources

Pillars of the AWS Well Architected Framework

AWS Well Architected Framework : Reliability Pillar

The ability of a system to:

- Recover from infrastructure or service failures
- Dynamically acquire computing resources to meet demand
- Mitigate disruptions such as:
  - Misconfigurations
  - Transient network issues

Test Recovery Procedures

In an on-premises environment, testing is often conducted to prove that the system works in a particular scenario; testing is not typically used to validate recovery strategies.

In the cloud, you can test how your system fails, and you can validate recovery procedures.

- Use automation to simulate different failures or to recreate scenarios that led to failures before.
- Expose failure pathways so you can test and rectify *before* a real failure scenario, thus reducing the risk of failure for untested components.

Automatically Recover From Failure

By monitoring a system for key performance indicators (KPIs), you can trigger automation when a threshold is breached.

- This allows for automatic notification and tracking of failures for automated recovery processes that work around or repair the failure.
- With more sophisticated automation, you can anticipate and remediate failures before they occur.

Horizontal Scaling To Increase Aggregate System Availability

- Replace one large resource with multiple small resources to reduce the impact of a single failure on the overall system.
- Distribute requests across multiple smaller resources to ensure that they don't share a common point of failure.

Stop Guessing Capacity

- A common cause of failure in on-premises systems is resource saturation, when the demands placed on a system exceed the capacity of that system (such as denial-of-service attacks).
- In the cloud, you can monitor demand and system utilization and automate the addition or removal of resources. This allows you to maintain the level of resources that will satisfy demand without over- or under-provisioning.

Key Services For Reliability

Amazon CloudWatch

Areas	Key Services	
Foundations	 AWS IAM	 Amazon VPC
Change management	 AWS CloudTrail	 AWS Config
Failure management	 AWS CloudFormation	

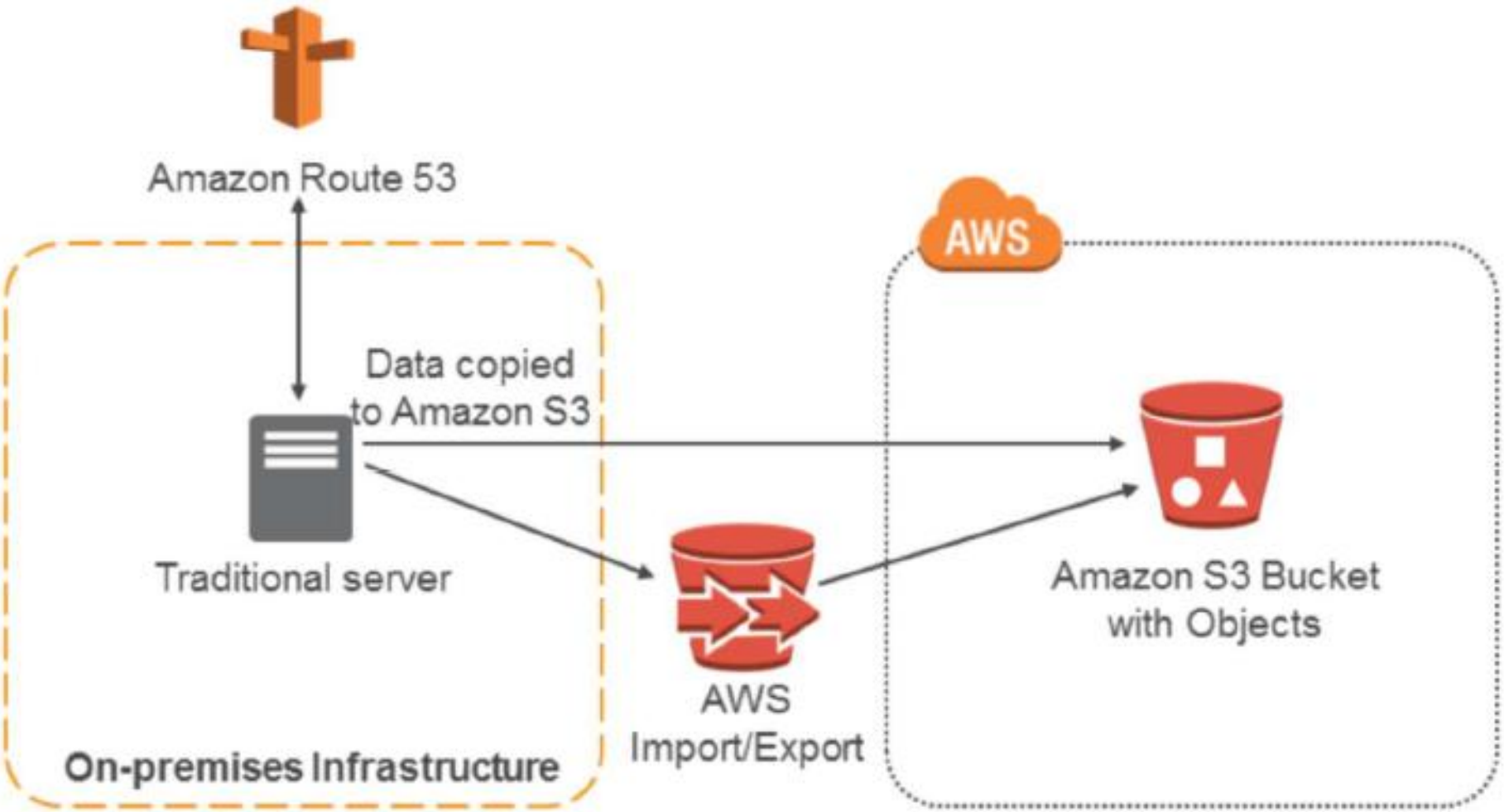
How to Make Infrastructure More Reliable?

Common Practices for Disaster Recovery (DR) On AWS

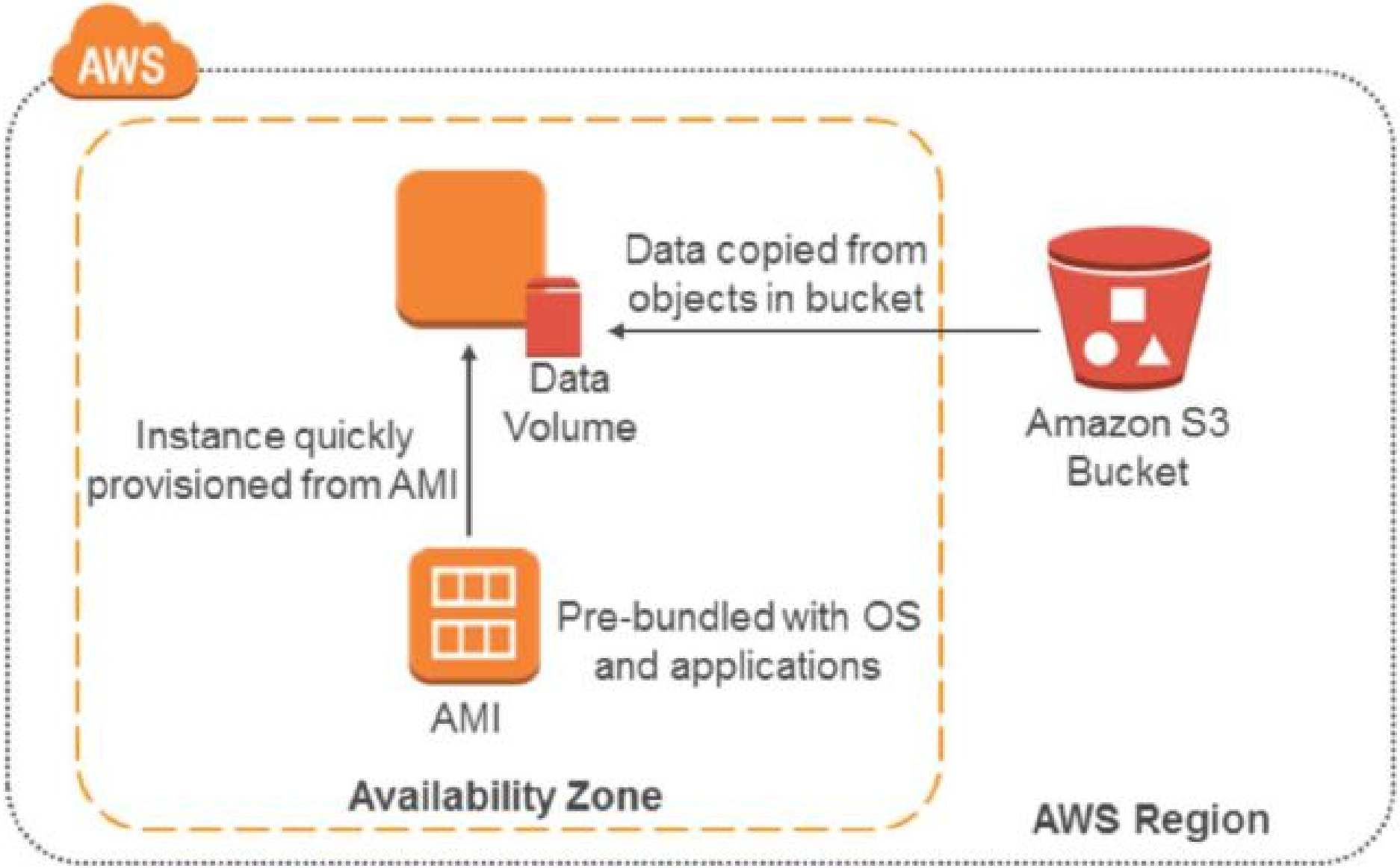
Example architectural patterns (sorted from highest to lowest RTO/RPO):

- Backup and Restore
- Pilot Light
- Fully Working Low-Capacity Standby
- Multi-Site Active-Active

Backup and Restore



Backup And Recovery



Backup And Recovery

Advantages

- Simple to get started.
- Extremely cost effective (mostly backup storage).

Preparation Phase

- Take backups of current systems.
- Store backups in Amazon S3.
- Describe procedure to restore from backup on AWS.
  - Know which AMI to use; build your own as needed.
  - Know how to restore system from backups.
  - Know how to switch to new system.
  - Know how to configure the deployment.

Backup And Recovery

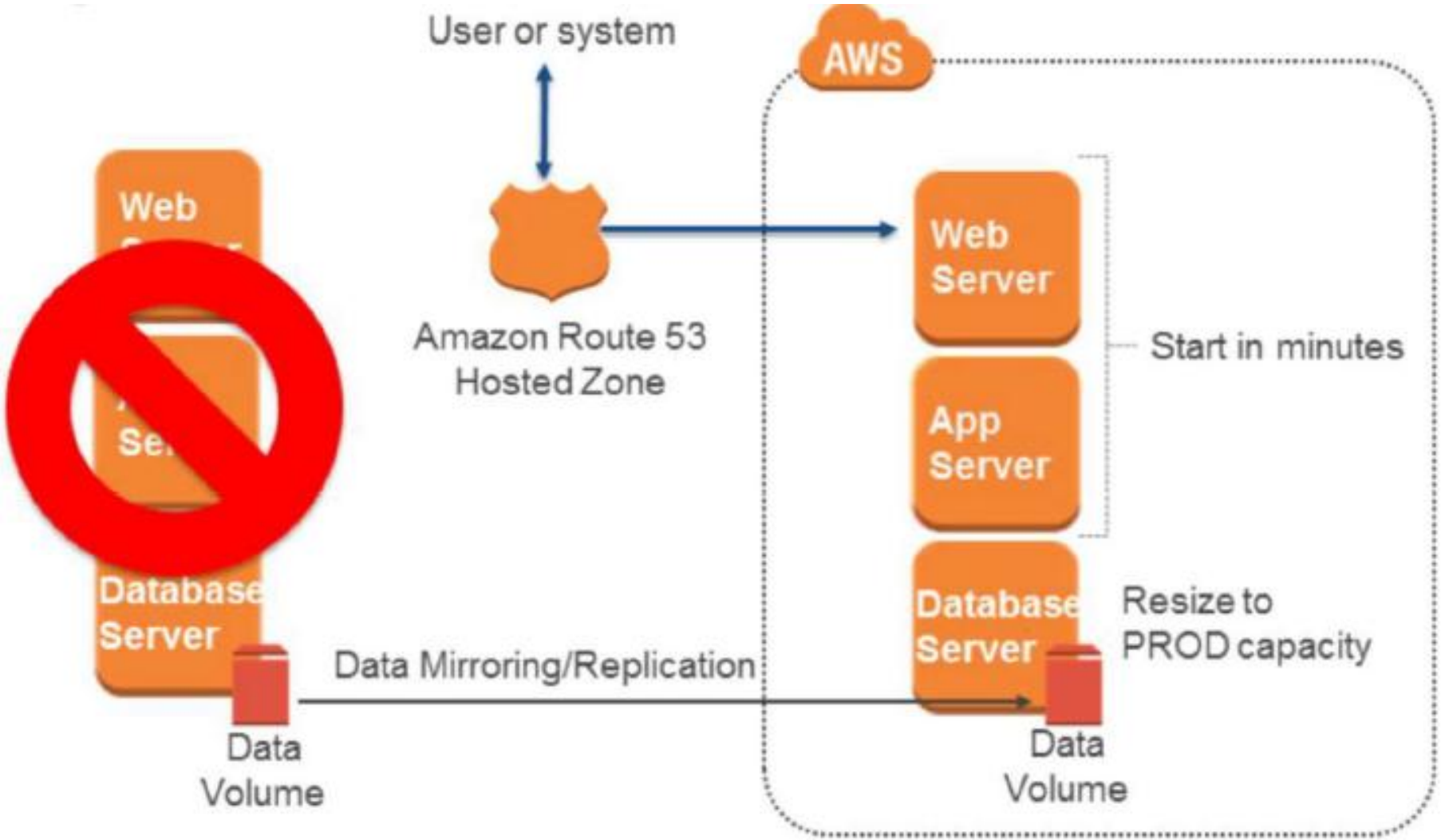
In case of disaster

- Retrieve backups from Amazon S3.
- Bring up required infrastructure.
  - Amazon EC2 instances with prepared AMIs, Elastic Load Balancing, etc.
  - Use Amazon CloudFormation to automate deployment of core networking.
- Restore system from backup.
- Switch over to the new system.
  - Adjust DNS records to point to AWS.

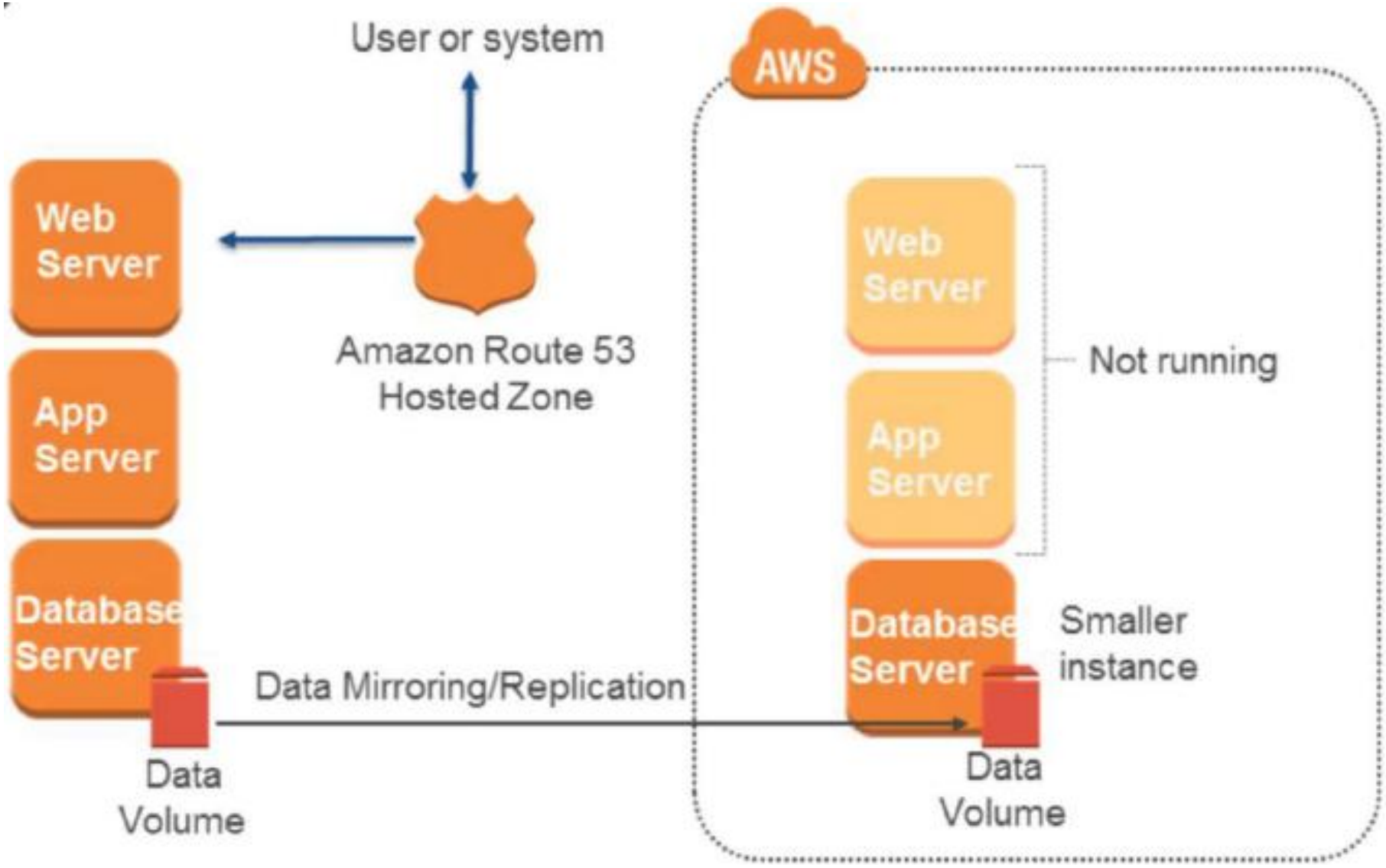
Objectives

- RTO: as long as it takes to bring up infrastructure and restore system from backups.
- RPO: time since last backup.

Pilot Light


User or system
AWS
Web
Server
Amazon Route 53
Hosted Zone
Web
Server
App
Server
Start in minutes
Database
Server
Database
Server
Resize to
PROD capacity
Data Mirroring/Replication
Data
Volume
Data
Volume

Pilot Light


User or system
AWS
Web Server
App Server
Database Server
Amazon Route 53 Hosted Zone
Web Server
App Server
Not running
Database Server
Smaller instance
Data Mirroring/Replication
Data Volume
Data Volume

Pilot Light

Advantage

- Very cost-effective (fewer 24/7 resources).

Preparation Phase

- Enable replication of all critical data to AWS.
- Prepare all required resources for automatic start.
  - AMIs, Network Settings, Elastic Load Balancing, etc.
- Reserved Instances.

Pilot Light

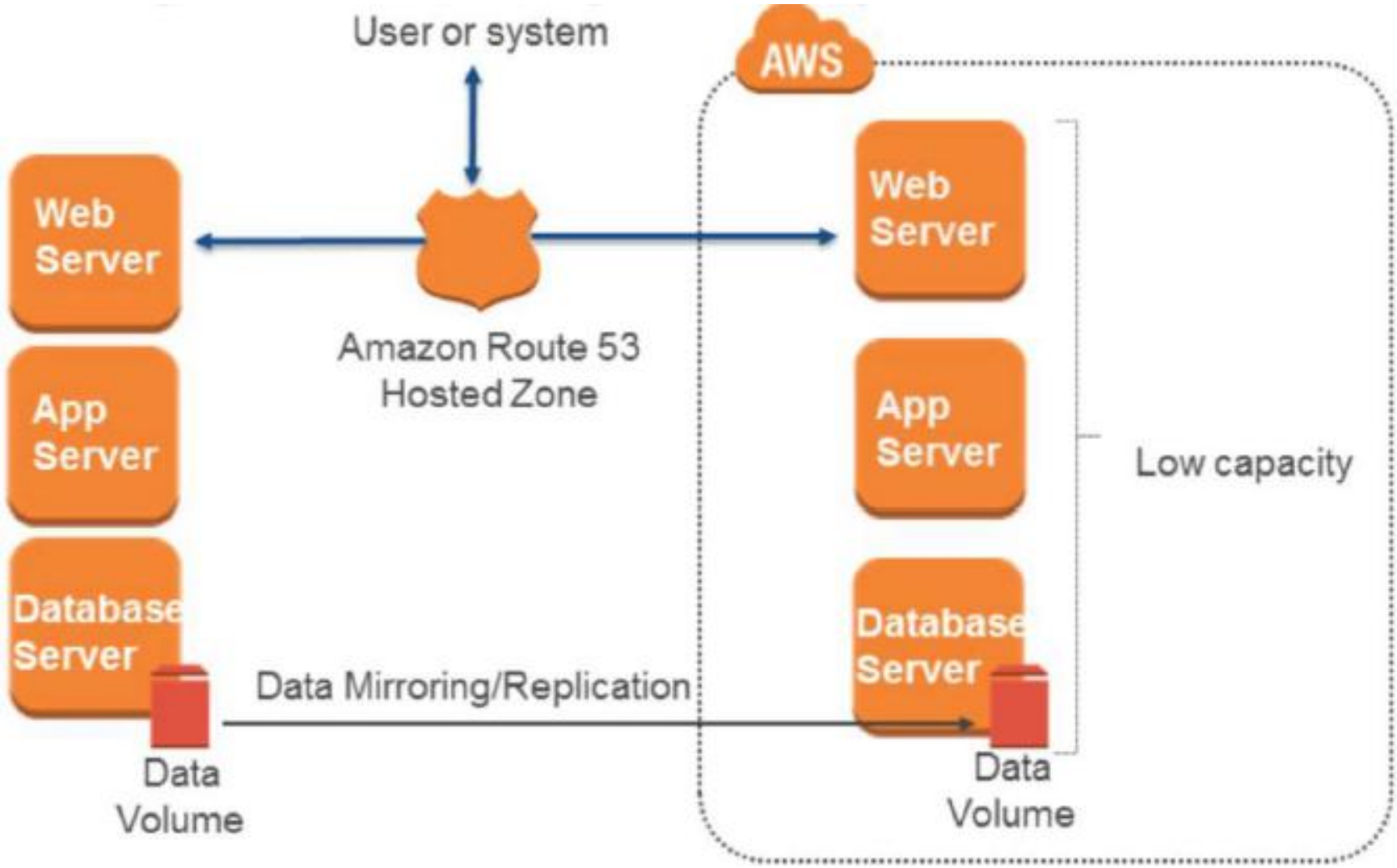
In case of disaster

- Automatically bring up resources around the replicated core data set.
- Scale the system as needed to handle current production traffic.
- Switch over to the new system.
  - Adjust DNS records to point to AWS.

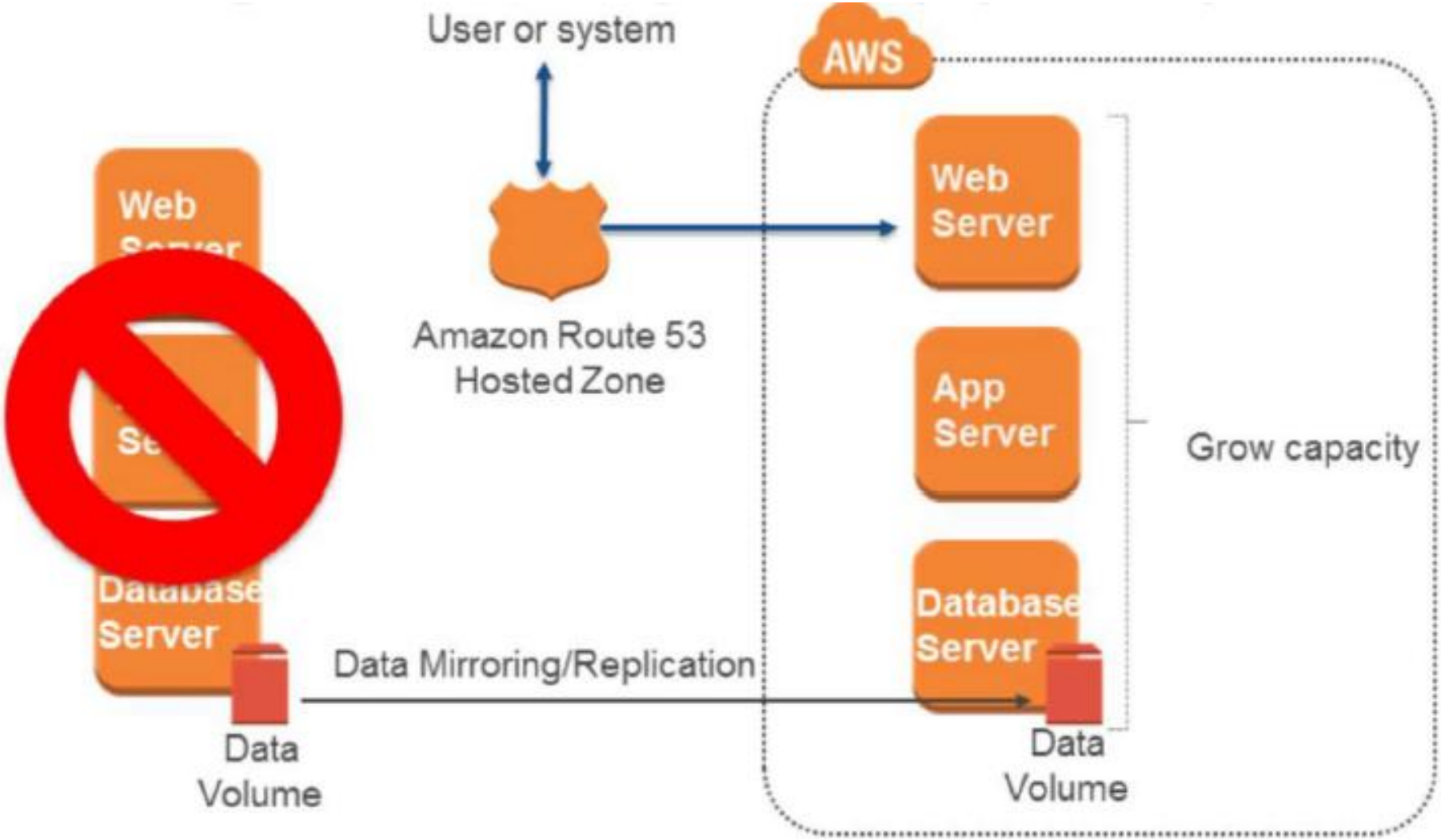
Objectives

- RTO: as long as it takes to detect need for DR and automatically scale up replacement system.
- RPO: depends on replication type.

Fully Working Low-Capacity Standby



Fully Working Low-Capacity Standby



Fully Working Low-Capacity Standby

Advantages

- Can take some production traffic at any time.
- Cost savings (IT footprint smaller than full DR).

Preparation

- Similar to Pilot Light.
- All necessary components running 24/7, but not scaled for production traffic.
- Best practice: continuous testing.
  - "Trickle" a statistical subset of production traffic to DR site.

Fully Working Low-Capacity Standby

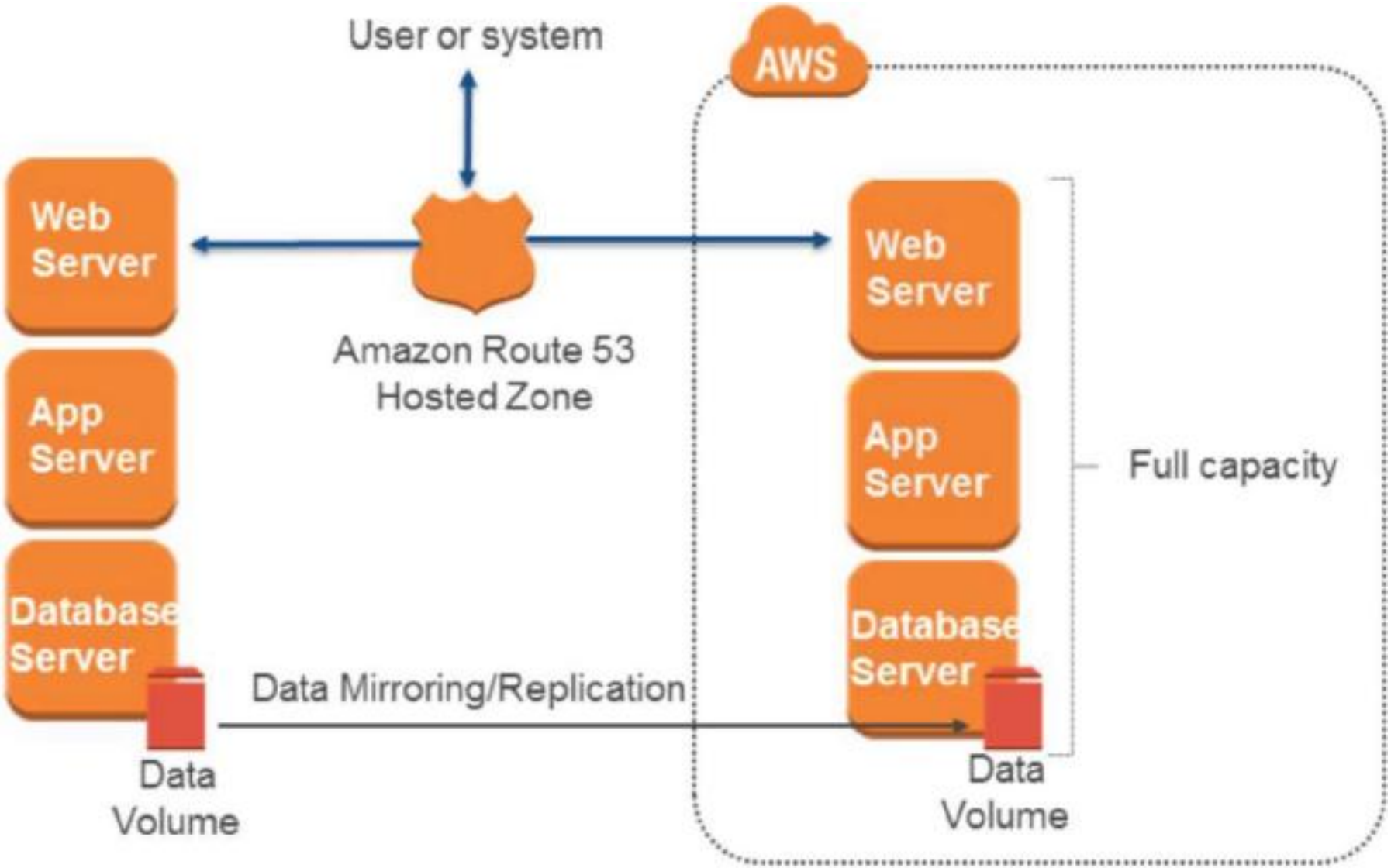
In case of disaster

- Immediately fail over most critical production load.
  - Adjust DNS records to point to AWS.
- (Auto) Scale the system further to handle all production load.

Objectives

- RTO: for critical load: as long as it takes to fail over; for all other load, as long as it takes to scale further.
- RPO: depends on replication type.

Multisite Active-Active



Multisite Active-Active

Advantages

- At any moment can take all production load.

Preparation

- Similar to Low-Capacity Standby.
- Fully scaling in/out with production load.

In Case of Disaster

- Immediately fail over all production load.

Objectives

- RTO: as long as it takes to fail over.
- RPO: depends on replication type.

Best Practices For Preparation

Start simple and work your way up

- Backups in AWS as a first step.
- Incrementally improve RTO/RPO as a continuous effort.

Check for any software licensing issues.

Exercise your DR Solution

- Practice "Game Day" exercises.
- Ensure backups, snapshots, AMIs, etc. are working.
- Monitor your monitoring system.

Advantages of DR in Cloud

- Various building blocks available.
- Fine control over cost vs. RTO/RPO tradeoffs.
- Ability to easily and effectively test your DR plan.
- Availability of multiple locations worldwide.

DR Example

Unilever Used AWS To Enable Better Disaster Recovery

Amazon S3

Backup data, snapshots, product and recipe media files stored durably and made available to all regions

Amazon EBS

EBS Snapshot Copy used to backup SQL Server on Amazon EC2 data volumes and restore them to the disaster recovery region when necessary

Amazon EC2

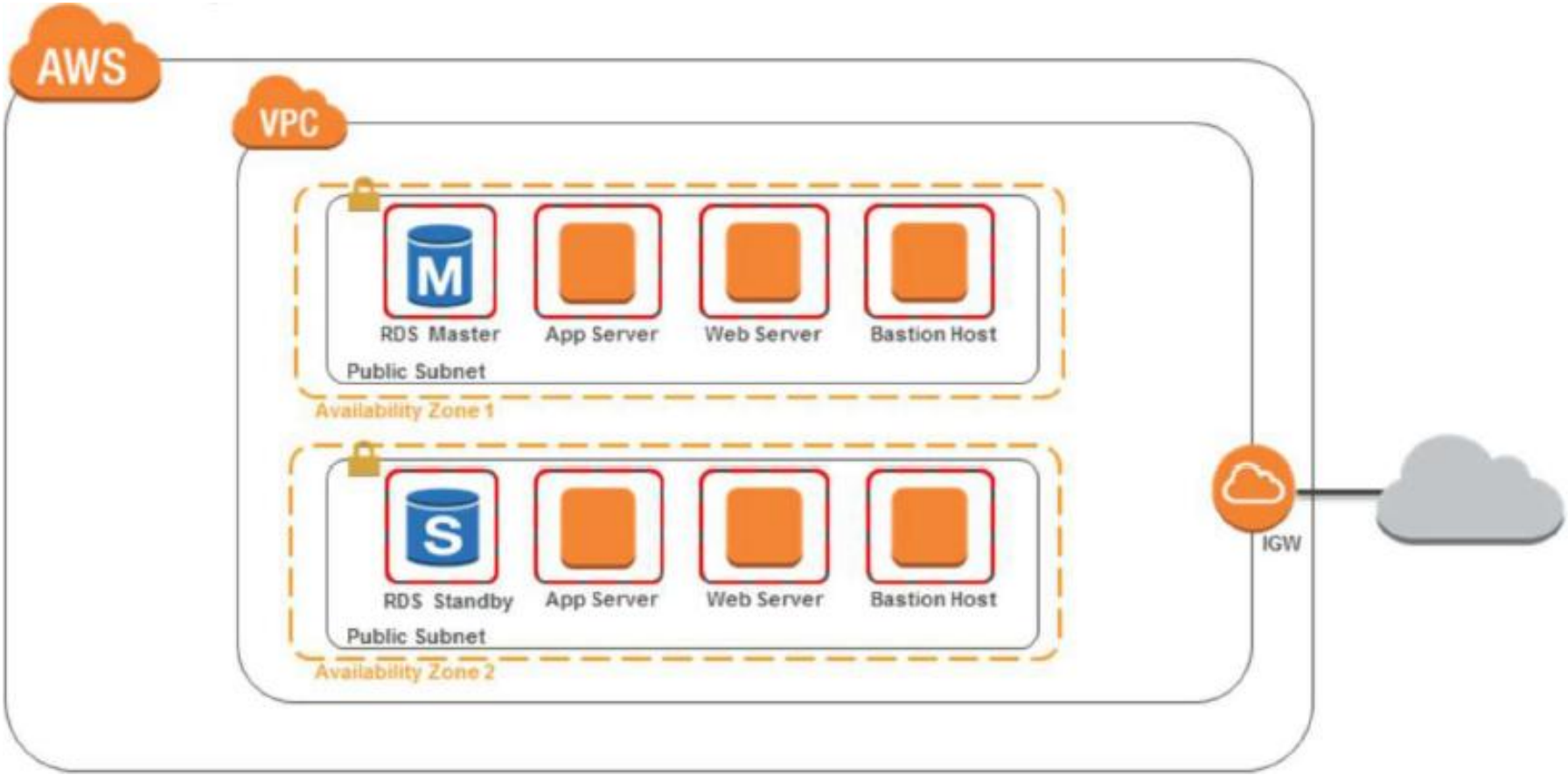
Created ~400 AMIs of Windows and Linux instances to provide flexible deployment across environments

In DR region, all instances except master SQL Server are kept on standby, activated when needed

Class Exercise –

Improving the Reliability of a Given Architecture

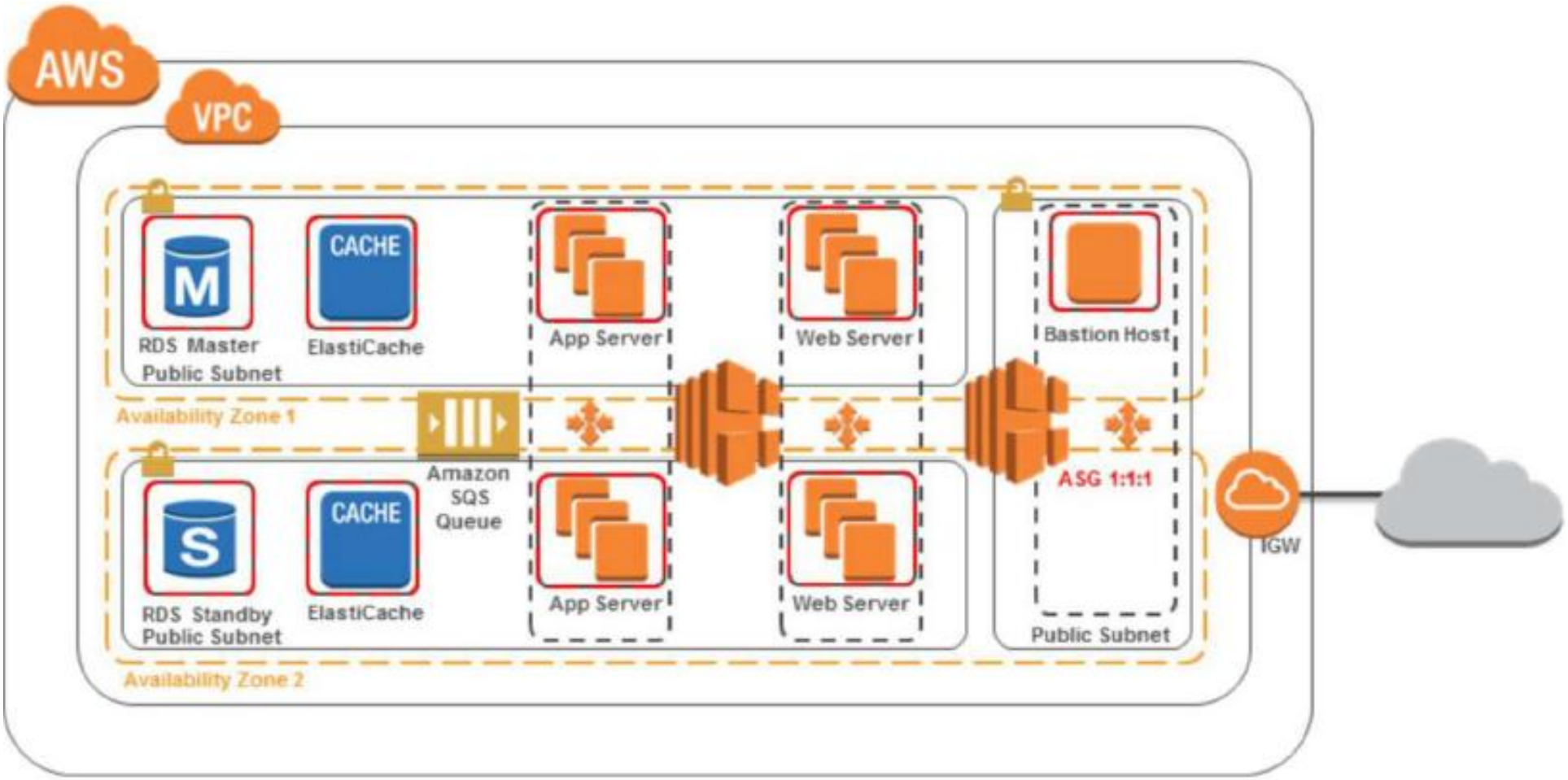
- Your team has just finished forklifting an application into the cloud. Due to a previous failure of the system, your CTO requests a Well Architected review of the system.
- Review the sample diagram and ask questions to identify where improvements can be made to increase the reliability of the system.
- Recommend reliability enhancements in accordance with the Well Architected Reliability pillar.



Tips

- Loose Coupling: Add internal ELB or SQS between Web and App Servers.
- Session Information: Offload to DynamoDB or ElastiCache.
- Would Read Replicas increase reliability?
- Auto Scaling Groups to ensure available workload
- 1:1:1 ASG on the Bastion Host

Proposed Solution



Questions

How are you managing AWS Service Limits for your account?

How are you managing AWS Service Limits for your account?

Best practice

- **Monitor and manage limits** by evaluating your potential usage on AWS, increase your regional limits appropriately, and allow planned growth in usage.
- **Set up automated monitoring** by implementing tools such as SDKs to alert you when thresholds are being approached.
- **Be aware of fixed service limits** that are unchangeable and architect around those.

How are you planning your network topology on AWS?

How are you planning your network topology on AWS?

- Use **highly available connectivity to AWS** with:
  - Multiple AWS DirectConnect circuits
  - Multiple VPN tunnels
  - AWS Marketplace appliances
- Use **highly available connectivity to the system** with:
  - Highly available load balancing and/or proxy
  - DNS-based solutions
  - AWS Marketplace appliances

Best Practices

- Use **non-overlapping private IP ranges** between all of your environments in and out of the cloud
- Size your **IP subnet allocation** to be large enough to accommodate future expansion

How does your system adapt to changes in demand?

How does your system adapt to changes in demand?

Best practice

- Use **automated scaling** features from services such as:
  - Amazon S3
  - Amazon CloudFront
  - Auto Scaling
  - Amazon DynamoDB
  - AWS Elastic Beanstalk
- Adopt a **load testing** methodology to measure if scaling activity will meet your application requirements.

How are you monitoring AWS resources?

How are you monitoring AWS resources?

Best practice

- **Monitor** your applications with:
  - Amazon CloudWatch
  - Third-party tools
- Configure **notifications** for when significant events occur.
- Use **automation** to take action when failure is detected.
- Perform frequent **reviews** of the system based on significant events to evaluate the architecture.

How are you executing change management?

How are you executing change management?

Best practice

- Perform **automated change management** for deployments and patching.

How are you backing up your data?

How are you backing up your data?

Best practice

- **Back up data** that is important with a required RPO using:
  - Amazon S3
  - Amazon EBS snapshots
  - Third-party software
- **Automate backups** using:
  - AWS features
  - AWS Marketplace solutions
  - Third-party software
- **Secure** and/or **encrypt** backups.
- Validate that the backup process implementation meets RTO and RPO through **periodic recovery testing**.

How does your system withstand component failures?

How does your system withstand component failures?

Best practice

- Use a **load balancer** in front of a pool of resources.
- Distribute applications across **multiple Availability Zones or Regions**.
- Use **automated healing** capabilities to detect failures and remediate.
- **Monitor** the health of your system continuously.
- Configure to receive **notifications** of any significant events.

How are you planning for recovery?

How are you planning for recovery?

Best practice

- **Define** RTO and RPO **objectives.**
- Establish a **disaster recovery** strategy.
- Avoid **configuration drift** by ensuring that AMIs are up-to-date at the DR site/region.
- Request an increase of **service limits** with the DR site to accommodate a failover.
- Regularly **test and validate disaster recovery** scenarios to ensure RTO and RPO are met.
- **Automate system recovery** using AWS and/or third-party tools.

AWS Well Architected Framework: Performance Efficiency

The ability to:

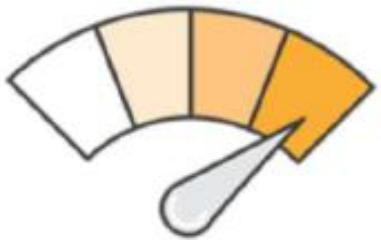
- Use computing resources efficiently to meet system requirements
- Maintain that efficiency as demand changes and technologies evolve

Democratize Advanced Technologies

Technologies that are difficult to implement can become easier to consume by pushing that knowledge and complexity into the cloud vendor's domain.

Instead of your IT team having to learn how to host and run a new technology, they can simply consume it as a service.

Example: NoSQL databases, media transcoding, and machine learning are all technologies that require expertise to implement. In the cloud, these technologies become services that your team can consume.



Go Global In Minutes

Easily deploy your system in multiple regions around the world with just a few clicks.

This allows you to provide lower latency and a better experience for your customers simply and at a minimal cost.

Use Serverless Architecture

In the cloud, serverless architectures remove the need for you to run and maintain servers to carry out traditional compute activities.

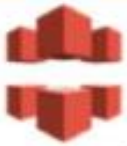
For example: storage services can act as static websites, removing the need for web servers, and event services can host your code for you.

This not only removes the operational burden of managing these servers, but also can lower transactional costs because these managed services operate at cloud scale.

Experiment More Often

With virtual and automated resources, you can quickly carry out comparative testing using different types of instances, storage, or configurations.

Key Services For Performance Efficiency

Areas	Key Services		
Compute	 Auto Scaling		
Storage	 Amazon EBS	 Amazon S3	 Amazon Glacier
Database	 Amazon RDS	 Amazon DynamoDB	
Space-time trade-off	 Amazon CloudFront		

How To Make Infrastructure Perform More Efficiently?

Choosing The Right EC2 Instance Type

- Evaluate the most important performance metrics for your application and choose the appropriate instance family and instance type.
- Avoid over-provisioning and under-provisioning.
- Change instance types and sizes as your needs change.
- Analyze if your application can scale out across multiple EC2 instances by design. Design applications that are resilient to reboot and relaunch to allow for scaling horizontally instead of vertically where possible.

Storage Comparison

	Instance Store	Amazon EBS	Amazon S3	Amazon Glacier
Average latency	ms	ms	ms, sec, min (~ size)	hrs
Data volume	4 GB – 48 TB	1 GiB – 16 TiB	No limit	No limit
Item size	Block storage	Block storage	5 TB max	40 TB max
Request rate	Very High	Very High	Low–Very High (no limit)	Very Low (no limit)
Cost/GB per month	Amazon EC2 instance cost	¢¢	¢	¢
Durability	Low	High	Very High	Very High

Database Comparison

	DynamoDB	Amazon RDS	Amazon Redshift
Average latency	ms	ms, sec	sec,min
Data volume	No limit	5 GB – 64 TB max (Max varies depending on engine)	1.6 PB max
Item size	KB (400 KB max)	KB (~rowsize)	KB (64 K max)
Request rate	Very High	High	Low
Cost/GB per month	¢¢	¢¢	¢
Durability	Very High	High	High

AWS ElastiCache

- Web service that makes it easy to deploy, operate, and scale an **in-memory cache** in the cloud.
- **Improves performance** of web applications by allowing you to retrieve information from fast, managed, in-memory caches, instead of relying entire on slower disk-based databases.
- **Supports Memcached and Redis** open-source in-memory caching engines.

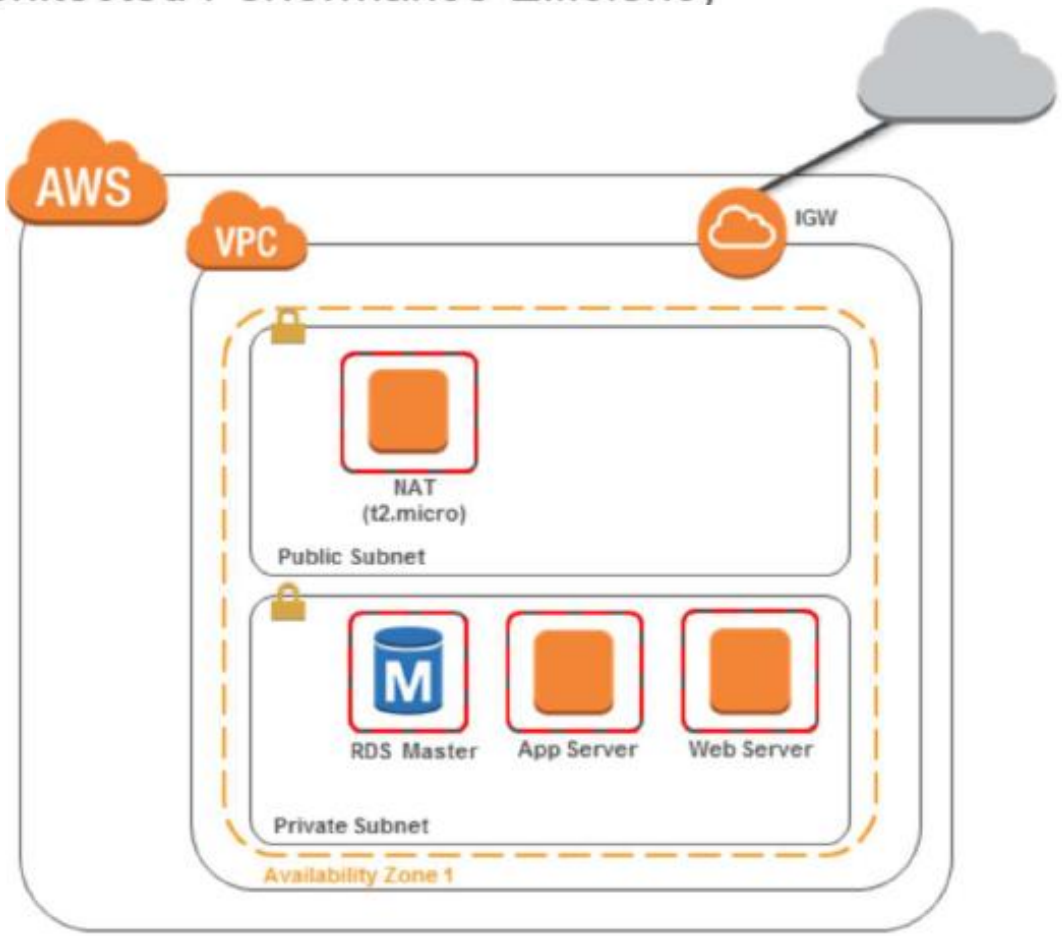
Amazon Kinesis

- Enables you to build custom applications that process or analyze **streaming data**.
- Can continuously capture and store **terabytes of data per hour** from **hundreds of thousands of sources**, such as
  - Website clickstreams
  - Financial transactions
  - Social media feeds
  - IT logs

Class Exercise :

Improve The Performance Efficiency for a Given Scenario

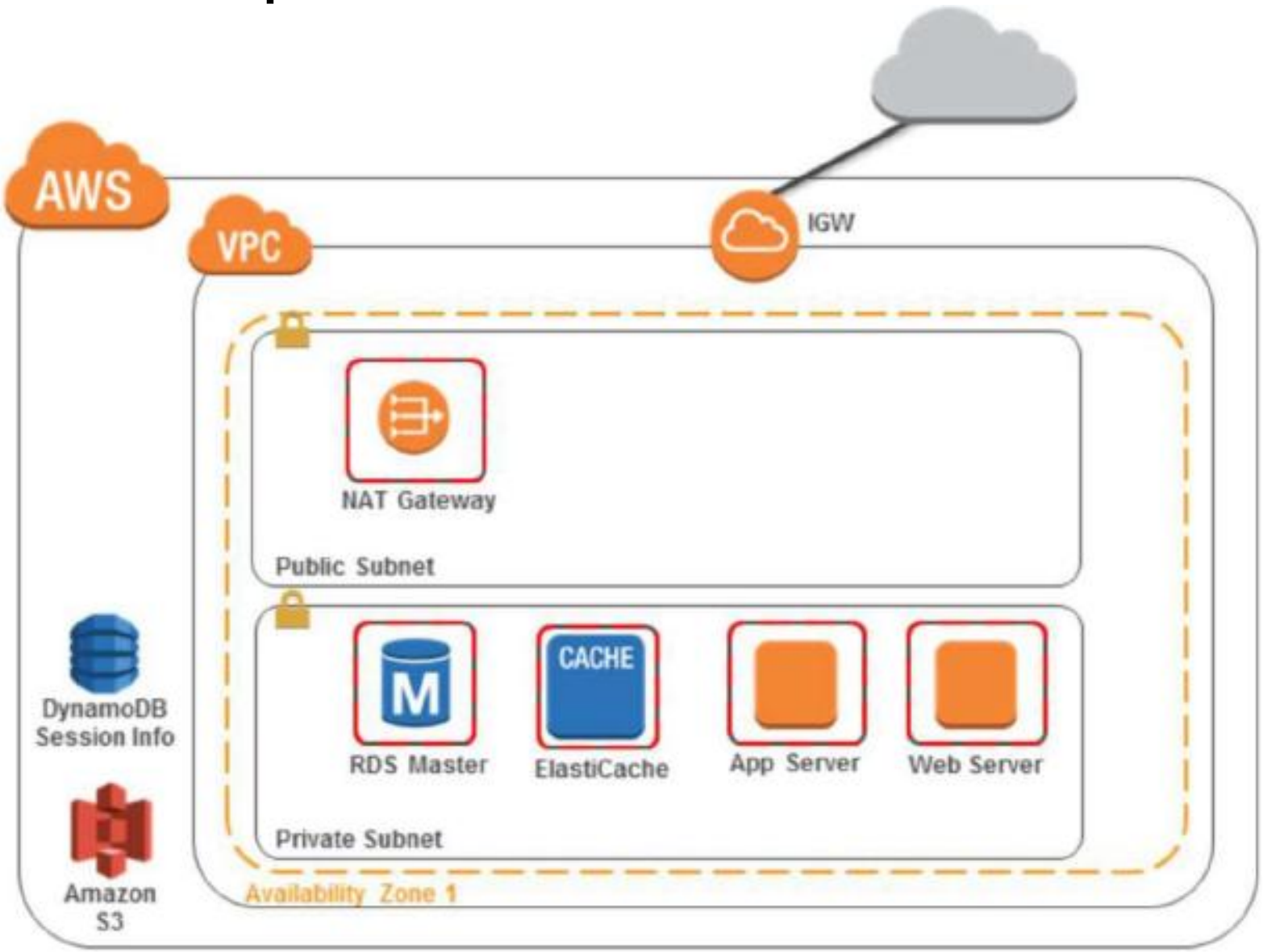
- Your small business website has been experiencing intermittent page load issues where users report significant load times.
- Review the diagram and ask questions to identify where improvements can be made to increase the Performance Efficiency of the system.
- Recommend performance efficiency enhancements in accordance with the Well Architected Performance Efficiency pillar.



Tips

- NAT box size – T2.micro with low throughput
- Managed NAT
- Why NAT? Why not ELB?
- Offloading static content to S3
- What is the site? Can it run out of S3?
- Disk I/O
- Is the page load due to throughput or due to Database latency?
- Read Replica?
- Geographic location and latency from region to User?
- Where are the users reports from? Route 53 Latency based routing and multiple regions?

Improved Solution



Questions

How do you select the appropriate instance types, storage solutions, database solutions, or proximity and caching solutions for your system?

How do you select the appropriate instance types, storage solutions, database solutions, or proximity and caching solutions for your system?

Best practice

- Select solutions based on:
  - Predicted resource needs
  - Required internal governance standards
  - Cost / budget
  - Benchmarking results
  - Load testing results
  - Guidance from AWS or from a member of the AWS Partner Network (APN)

How do you ensure that you continue to have the most appropriate instance types, storage solutions, database solutions, and proximity and caching solutions as new services and features are launched?

How do you ensure that you continue to have the most appropriate instance types, storage solutions, database solutions, and proximity and caching solutions as new services and features are launched?

Best practice

- **Review** cyclically and reselect new services and features based on predicted resource needs
- **Benchmark** and **load test** after each new service or feature is released, and use that information to make the **best selection** based on a calculation of performance/cost.

How do you monitor your instances, storage solutions, databases, and proximity and caching solutions to ensure they are performing as expected?

How do you monitor your instances, storage solutions, databases, and proximity and caching solutions to ensure they are performing as expected?

Best practice

- Monitor instances with:
  - **Amazon CloudWatch**
  - **Third-party tools**
- Perform a **periodic review** of your monitoring dashboards
- Use **alarm-based notifications** to automatically alert you if metrics are out of safe bounds.
- Use **trigger-based actions** to cause automated actions to remediate or escalate issues.

How do you ensure that the capacity and throughput of your instances, storage solutions, databases and proximity and caching solutions match demand?

How do you ensure that the capacity and throughput of your instances, storage solutions, databases and proximity and caching solutions match demand?

Best practice

- **React** based on manually reviewing metrics
- **Plan** future capacity and throughput based on metrics and/or planned events
- **Automate** against metrics
- Perform a **periodic review** of cache usage and demand over time
- **Monitor** usage and demand over time
- Use the following for **automatic management**:
  - Scripting and tools
  - Auto Scaling

AWS Well Architected Framework: Cost Optimization

The ability to avoid or eliminate:

- Unneeded cost
- Suboptimal resources

Best Practice : Optimize For The Cost

Take advantage of AWS's flexible platform to increase your cost efficiency.

Ensure that your resources are sized appropriately, that they scale in and out based on need, and that you're taking advantage of different pricing options.

Things to consider:

- Are my resources the right size and type for the job?
- What metrics should I monitor?
- How do I make sure that resources not in use are turned off?
- How often will I need to use this resource?
- Can I replace any of my servers with managed services?

Principles of Cost Optimization

Transparently Attribute Expenditure

The cloud helps to:

- Identify the cost of a system and attribute IT costs to individual business owners.
- Identify return on investment (ROI). Consequently, this gives owners an incentive to optimize their resources and reduce costs.

Use Managed Services to Reduce TCO

Managed services remove the operational burden of maintaining servers for tasks such as sending email or managing databases.

Because managed services operate at cloud scale, they can offer a lower cost per transaction or service.

Trade Capital for Operating Expense

Do not invest heavily in data centers and servers before you know how you're going to use them.

Pay only for the computing resources you consume, when you consume them.

- Example: Development and test environments are typically only used for eight hours during the working week. Stop these resources when not in use, for a potential cost savings of 75% (40 hours versus 168 hours).

Benefit From Economies Of Scale

Achieve a lower variable cost in the cloud than you could on your own because AWS can achieve higher economics of scale.

Hundreds of thousands of customers are aggregated in the AWS cloud, which translates to lower pay-as-you-go prices.

Stop Spendings on IT & Data Center Operations

AWS does the difficult work of racking, stacking, and powering servers.

You can focus on your customers and business projects instead of on IT infrastructure.

Key Services For Cost Optimization


Cost Allocation Tags

Areas	Key Services	
Matched supply and demand	 Auto Scaling	
Cost-effective resources	 Spot and Reserved Instances	 AWS Trusted Advisor
Expenditure awareness	 Amazon CloudWatch	 Amazon SNS
Optimizing over time	 AWS Blog & What's New	 AWS Trusted Advisor

How to Optimize Infrastructure Costs?

AWS Free Tier

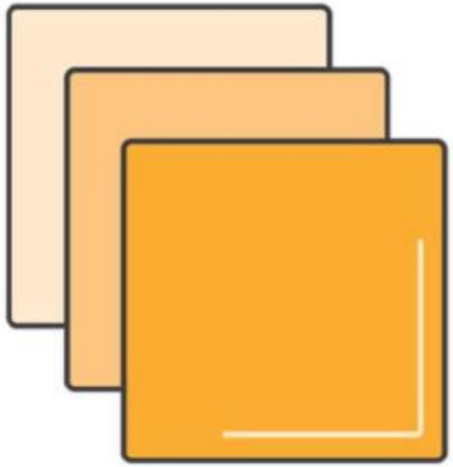
- Get hands-on experience with AWS Cloud Services.
- Use services for 12 months following account sign-up for free at certain usage limits.

- Other Free Options:
  - AWS Free Tier Eligible Software
  - AWS Free Tier Non-Expiring Offers

EC2 Purchasing Options

- On-Demand Instances
- Spot Instances
- Reserved Instances
- Dedicated Instances
- Dedicated Hosts

EC2 On-Demand



- Pay for compute capacity by the hour
- No long-term commitments
- No upfront payments
- Increase or decrease your compute capacity depending on the demands of your application
- Recommended for:
  - Users that want the low cost and flexibility of Amazon EC2 without an up-front payment or long-term commitment
  - Applications with short term, spiky, or unpredictable workloads that cannot be interrupted
  - Applications being developed or tested on Amazon EC2 for the first time

EC2 Spot Instances

- Bid for unused AWS capacity.
- Prices controlled by AWS based on supply and demand
- Termination Notice provided 2 minutes prior to termination, stored in metadata
- Best approach to temporary requests for large numbers of servers.

Spot Use Cases

Use Case	Types of Applications
Batch processing	Generic background processing (scale out computing)
Web/data crawling	Analyze data
Financial	Hedge fund analytics, energy trading, etc.
Elastic Map Reduce	Hadoop (large data processing)
Grid computing	Scientific trials/simulations in chemistry, physics, and biology
Transcoding	Transform videos into specific formats
Gaming	Back-end servers for Facebook games
Testing	Scale to large server pool to test software, websites, etc.

Spot Market Considerations (Vimeo)

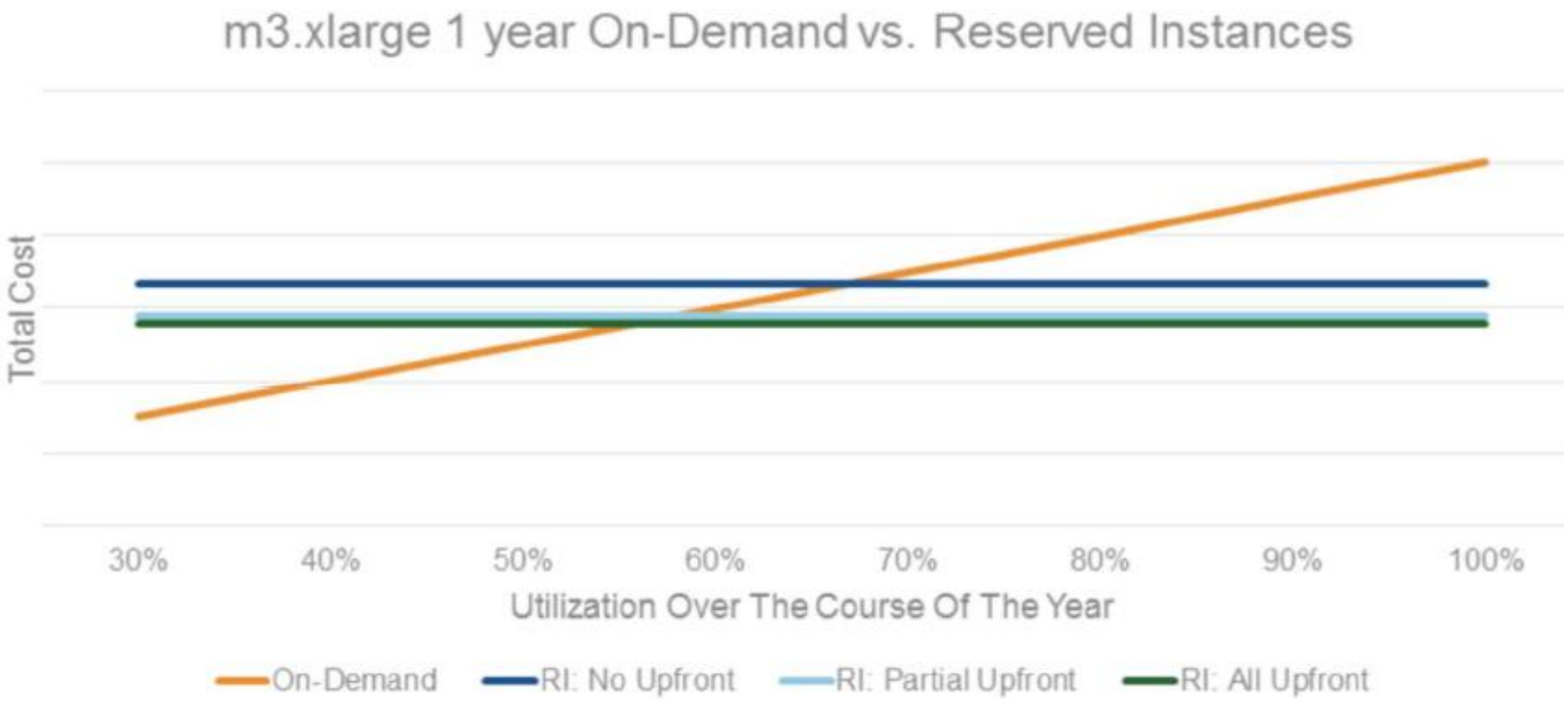
- Never bid more than threshold (80% of on-demand price).
- No more than 10 open spot requests at any time.
- Bid 10% more than the average price over last hour.
- Use spots for low-priority and less time-critical jobs.
- Have more retries for jobs running on spots.
- Watch out for open spot requests (add expiry to your requests).
- Billed hour forward unless terminated by AWS.
- For **long running jobs**, either bid higher on spot or use on-demand instances.
- Fail over to on-demand when spot market is saturated.

EC2 Reserved Instances (RI)

Up to 75% discount compared to On-Demand Instance pricing

- No Upfront
  - Access a Reserved Instance without an upfront payment.
  - Discounted effective hourly rate for every hour within the term, regardless of usage.
  - 1-year reservation available.
- Partial Upfront
  - Part of the Reserved Instance must be paid at the start of the term.
  - Discounted effective hourly rate for the remainder of the term, regardless of usage.
  - 1-year or 3-year reservations available.
- All Upfront
  - Full payment made at the start of the term.
  - No other costs incurred for the remainder of the term, regardless of usage.
  - 1-year or 3-year reservations available.

On-Demand vs RI Break-Even Points



RI Marketplace

- Flexibility
  - Sell your unused Amazon EC2 Reserved Instances
  - Buy Amazon EC2 Reserved Instances from other AWS customers
  - As your needs change, change your Reserved Instances
- Diverse term and pricing options
  - Shorter terms
  - Opportunity to save on upfront pricing
- Identical capacity reservations

RI: Scheduled Instances

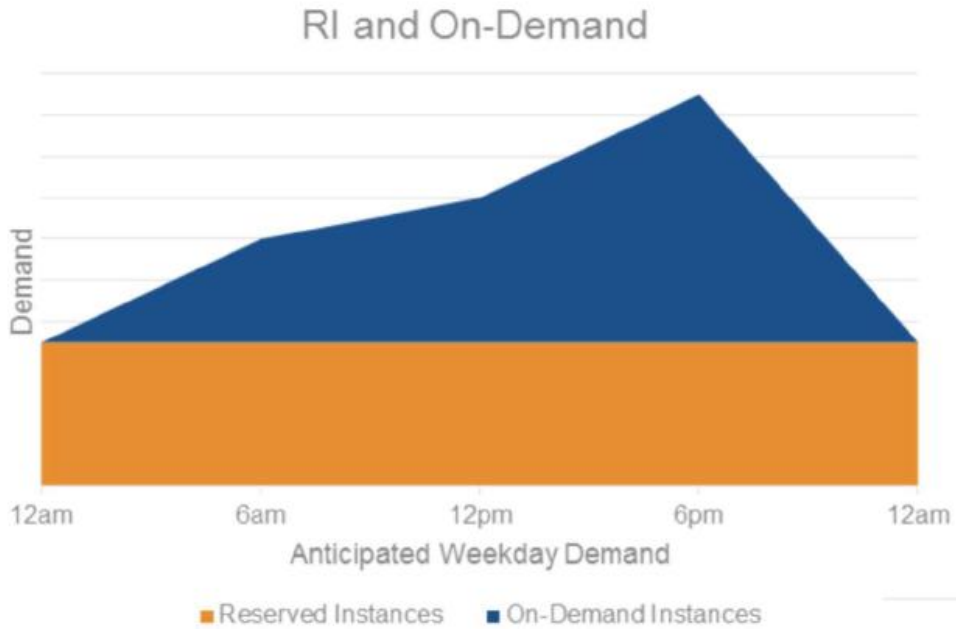
Reserve instances for predefined blocks of time on a recurring basis for a one-year term, with prices that are generally 5 to 10% lower than the equivalent On-Demand rates.

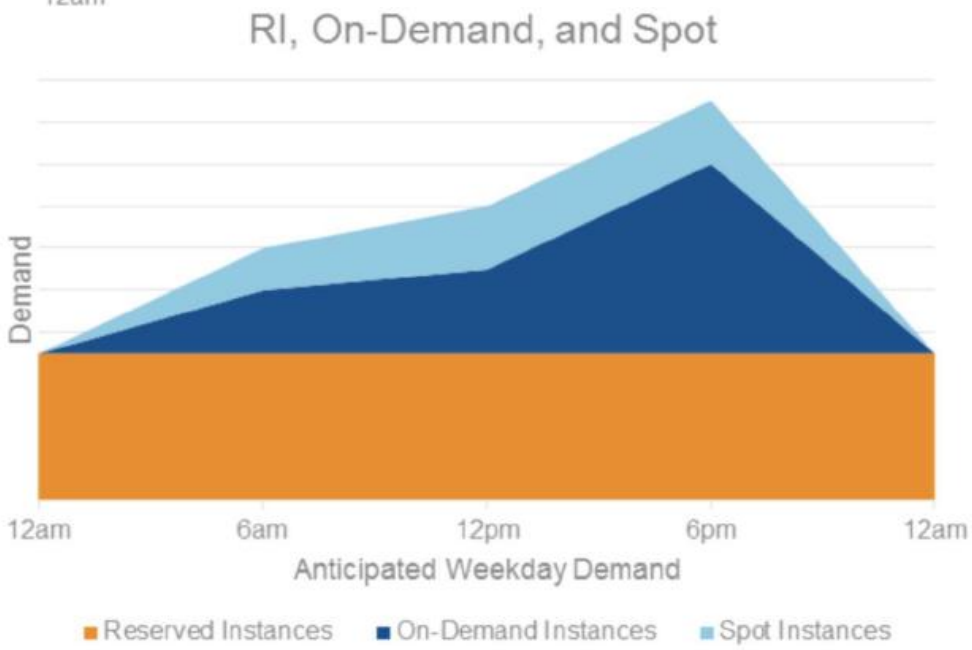
- Run reserved instances on a daily, weekly, or monthly basis.
- After you purchase your reserved instances, they are available to launch during the time windows that you've specified.
- Can be launched via the AWS Management Console, AWS CLI, AWS Tools for Windows PowerShell, and the `RunScheduledInstances` function.

RI: Scheduled Instances – Use Cases

- A bank performs Value at Risk calculations every weekday afternoon.
- A phone company does a multi-day bill calculation run at the start of each month.
- A trucking company optimizes routes and shipments on Monday, Wednesday, and Friday mornings.
- An animation studio performs a detailed, compute-intensive 3D rendering every night.

Leveraging EC2 Pricing Models





Blended Approach

- Choose instance type that matches requirements.
  - Start with memory requirements and architecture type (32-bit or 64-bit).
  - Then choose the closest number of virtual cores required.
- Scale across Availability Zones.
  - Smaller increments give more granularity for deploying to multiple AZs.
- Start with on-demand and then assess utilization for RIs.

Dedicated Instances vs Dedicated Hosts

Characteristic	Dedicated Instances	Dedicated Hosts
Dedicated physical servers	✓	✓
Per instance billing (subject to a $2 per region fee)	✓	✗
Per host billing	✗	✓
Visibility of sockets, cores, host ID	✗	✓
Affinity between a host and instance	✗	✓
Targeted instance placement	✗	✓
Automatic instance placement	✓	✓
Add capacity using an allocation request	✗	✓

Dedicated Instances

- Single customer tenancy on compute hardware.
- Physically isolated at the host hardware level:
  - From other instances that are not dedicated instances.
  - From instances that belong to other AWS accounts.
- Useful for workloads with **unique** compliance requirements.
  - Example: HIPAA PHI
- May not be needed unless physical isolation is required.

Dedicated Hosts

- A dedicated host is a physical EC2 server with instance capacity **fully dedicated** for your use.
- Dedicated hosts can help you:
  - Use your existing **server-bound licenses**
  - Meet **compliance** and regulatory requirements

Tools

AWS Trusted Advisor (TA)

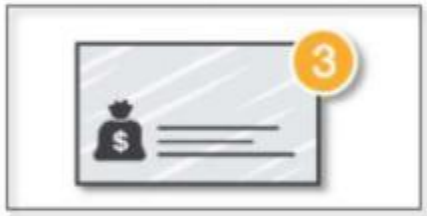
- AWS Trusted Advisor provides best practices (checks) in:
  - Cost Optimization
  - Security
  - Fault Tolerance
  - Performance Improvement
- Best practices available to all customers:
  - Service Limits
  - Security Groups – Specific Ports Unrestricted
  - IAM Use
  - MFA on Root Account

Over 40 Trusted Advisor checks available with Business and Enterprise Support plans

AWS TA – Features and Functionalities

AWS Trusted Advisor provides a suite of features for you to customize recommendations and to proactively monitor your AWS resources.

Notifications



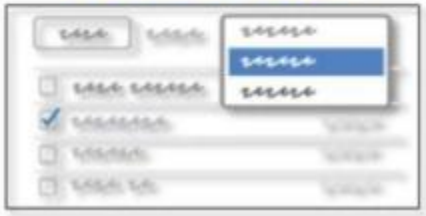
Access Management

AWS Support API

Action Links Beta

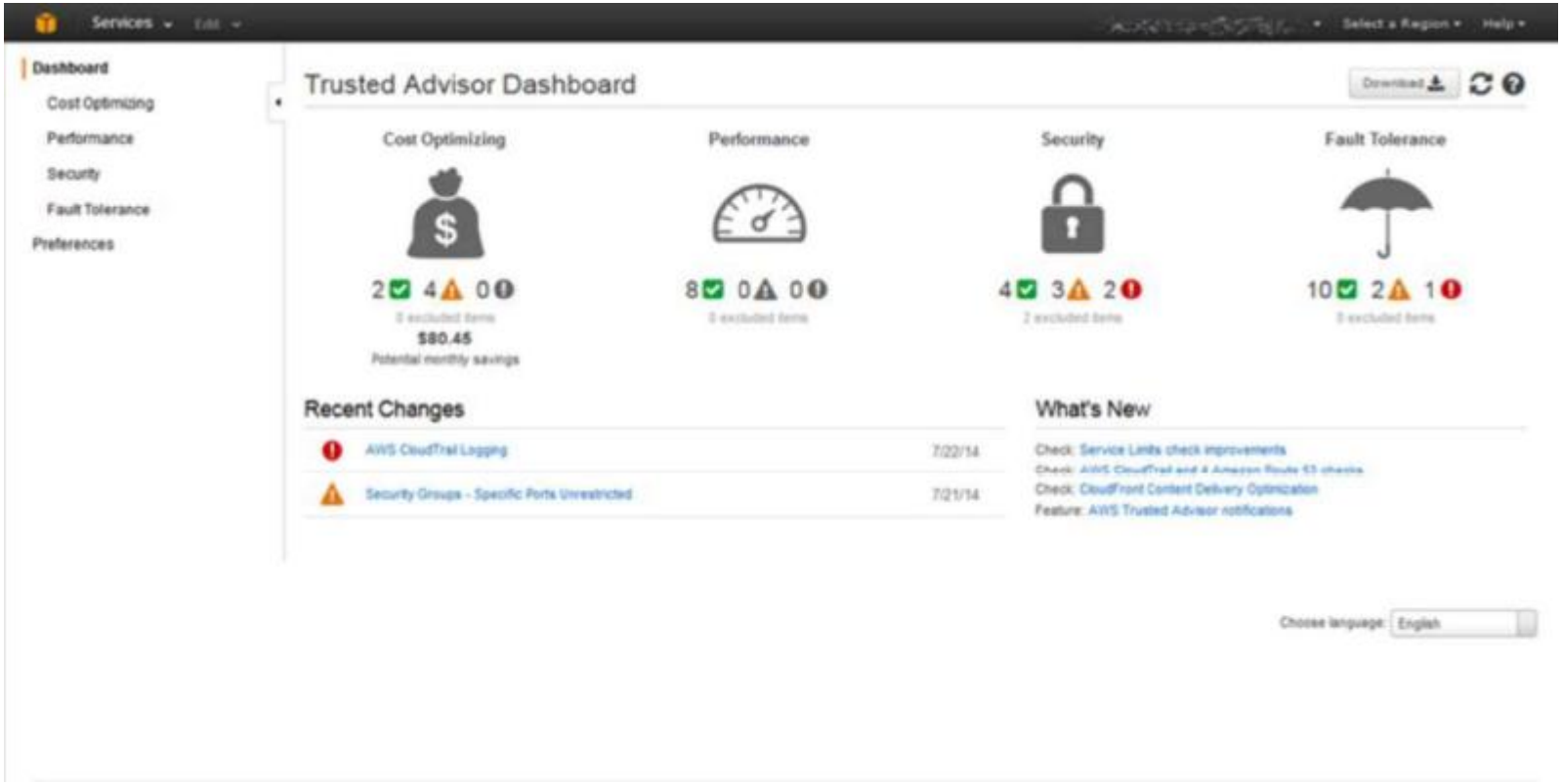
Recent Changes

Exclude Items



5-Min Refresh

AWS TA – Dashboard

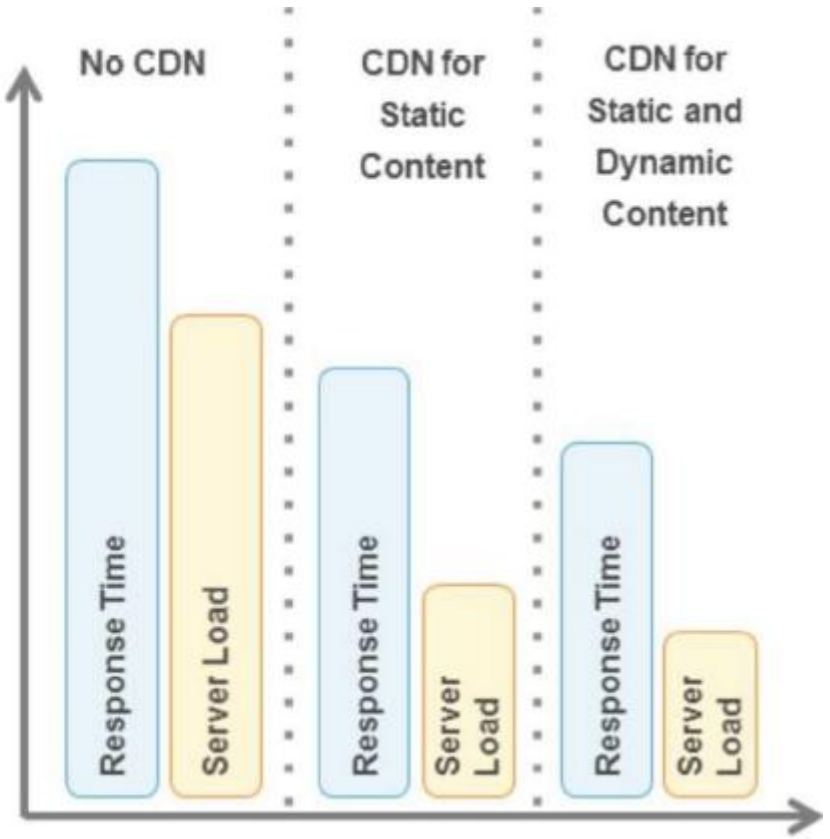


Cost Optimization with Caching

Serving Content With Caching

The more you can offload, the less infrastructure you need to maintain, scale, and pay for.

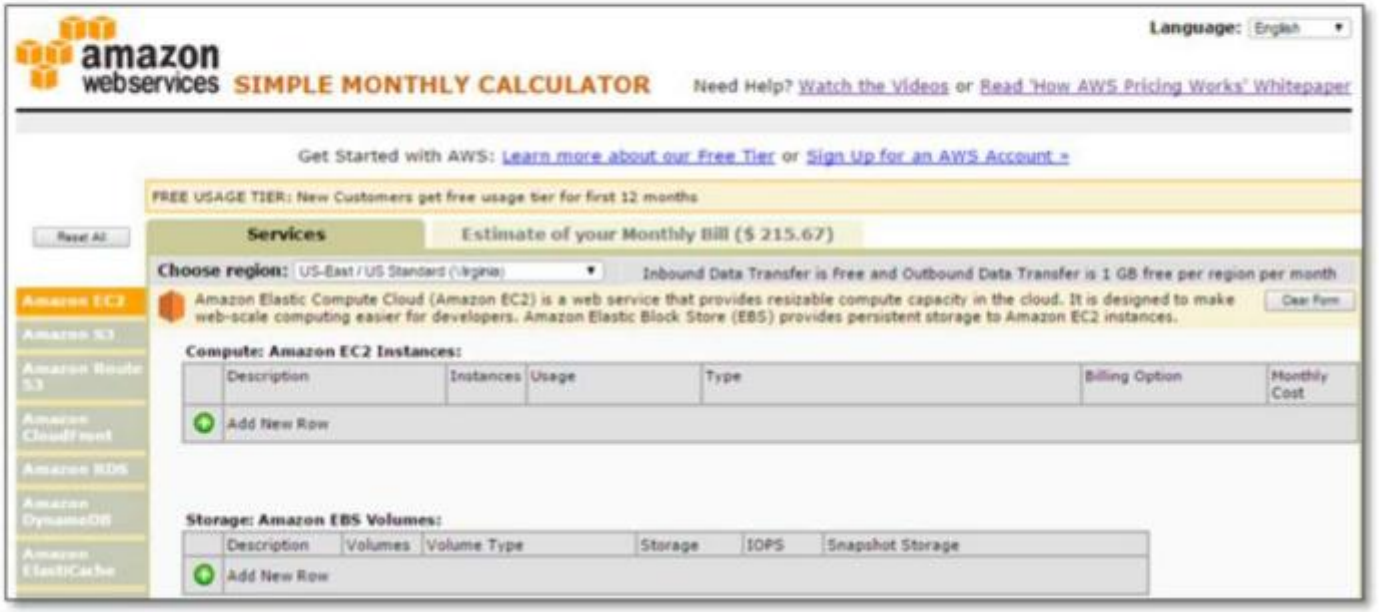
- Offload popular traffic to Amazon CloudFront and Amazon S3.
- Introduce caching.



Cost Calculation Tools

AWS Simple Monthly Calculator

Estimate the cost of running your application or solution in the AWS cloud based on usage



https://calculator.s3.amazonaws.com/index.html

AWS TCO Calculator

- Estimate **cost savings** when using AWS
- Use a **detailed set of reports** that can be used in executive presentations
- Modify **assumptions** that best meet your business needs

1. Describe your infrastructure in four steps, or enter detailed configurations

2. Get an instant summary report

3. Download a full report including detailed cost breakdowns

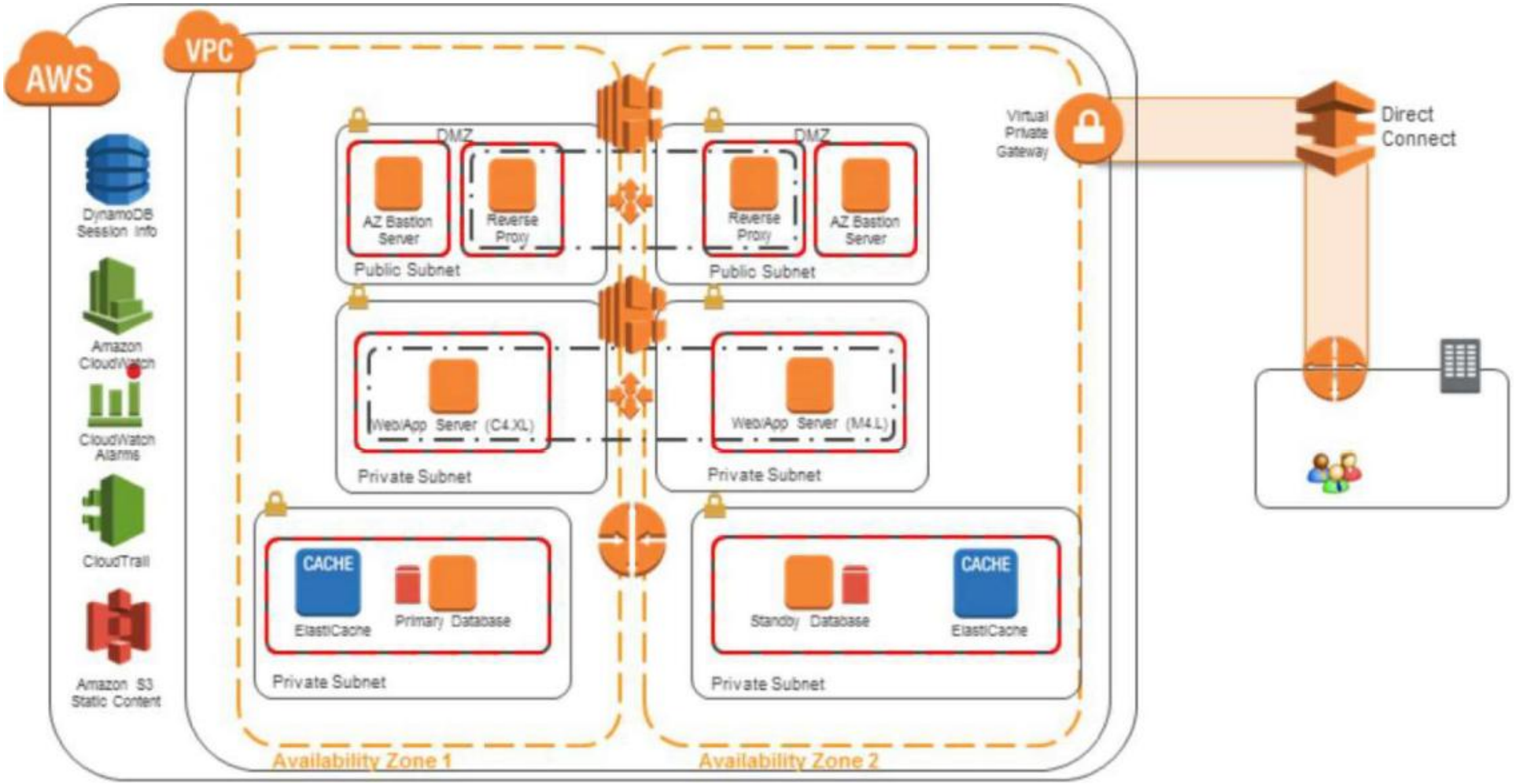
https://awstcocalculator.com/

Class Exercise – Cost Optimization

Scenario

- Your team performed a security, reliability, and performance assessment and put in every option they thought would increase performance. Your Cost Explorer indicates that your forecast is higher than your budget.
- Review the diagram and ask questions to identify where improvements can be made to improve the Cost Optimization of the system.
- Recommend ways to redesign the solution in accordance with the Well Architected Cost Optimization pillar.

Scenario


AWS
VPC
DynamoDB
Session Info
Amazon
CloudWatch
CloudWatch
Alarms
CloudTrail
Amazon S3
Static Content
DMZ
AZ Bastion
Server
Reverse
Proxy
Public Subnet
DMZ
Reverse
Proxy
AZ Bastion
Server
Public Subnet
Web/App Server (C4.XL)
Private Subnet
Web/App Server (M4.L)
Private Subnet
CACHE
ElastiCache
Primary Database
Private Subnet
Standby Database
CACHE
ElastiCache
Private Subnet
Availability Zone 1
Availability Zone 2
Virtual
Private
Gateway
Direct
Connect

Assessment Tips

- Unbalanced Instance Sizes?
- Why a C4 for a web server?
- Why run large Web/App servers behind small reverse proxies?
- Why DIY Databases and not RDS?
- Why ElastiCache? What purpose does it serve?
- Memcached or Redis?
- How many nodes in ElastiCache Cluster?
- Bandwidth of Direct Connect?
- Throughput of DynamoDB
- CloudWatch – Detailed monitoring or default interval?
- No Glacier – Could there be cost savings of moving some data off S3 and on to Glacier?
- Cost of CloudTrail versus cost of no Auditing
- Why two load balancers? Could SQS be used?
- Why two bastion hosts?
- Could any of this be done with Lambda?

Questions

How do you make sure your capacity matches but does not substantially exceed what you need?

How do you make sure your capacity matches but does not substantially exceed what you need?

Anti-pattern

- Over-utilization
- Over-provisioning

Best practice

- Approaches:
  - Demand-based using Auto Scaling
  - Queue-based using Amazon SQS
  - Time-based using scheduling
- Appropriately provisioned

How are you optimizing your usage of AWS services?

How are you optimizing your usage of AWS services?

Best practice

- Service-specific optimizations, such as:
  - Minimize I/O for Amazon EBS
  - Avoid uploading too many small files into Amazon S3
  - Use Spot instances extensively for Amazon EMR

Have you selected the appropriate pricing model to meet your cost targets?

Have you selected the appropriate pricing model to meet your cost targets?

Best practice

- Use **Spot instances** for select workloads
- Perform regular **analysis of usage** and purchase Reserved Instances accordingly
- Factor in cost when choosing a **region**
- **Automate** turning off unused instances when not needed
- Sell Reserved Instances you no longer need on the **Reserved Instance Marketplace**, and purchase others

Are there managed services (higher-level services than Amazon EC2, Amazon EBS, Amazon S3) you can use to improve your ROI?

Are there managed services (higher-level services than Amazon EC2, Amazon EBS, Amazon S3) you can use to improve your ROI?

Best practice

- Consider **other application level services**:
  - Amazon Simple Queue Service (SQS)
  - Amazon Simple Notification Service (SNS)
  - Amazon Simple Email Service (SES)
- Achieve the benefits of **standardization** and **cost control** using:
  - AWS CloudFormation templates
  - AWS Elastic Beanstalk
  - AWS OpsWorks
- **Consider appropriate databases:**
  - Amazon RDS (PostgreSQL, MySQL, Microsoft SQL Server, Oracle, MariaDB, Amazon Aurora)
  - Amazon DynamoDB

How are you monitoring usage and spending?

How are you monitoring usage and spending?

- **Tag all resources** to be able to correlate changes in your bill to changes in your infrastructure and usage
- Have a standard process to load and interpret **Detailed Billing Reports**
- Have a plan for both usage and spending in designing a **cost-efficient architecture**
- Use **AWS Cost Explorer**
- **Monitor** usage and spend regularly using Amazon CloudWatch or a third-party provider (Cloudability, CloudCheckr)
- Set up **notifications** to let key members of your team know if your spending moves outside of defined limits.
- Use a **finance-driven charge back method** to allocate instances and resources to cost centers (such as tagging)

Do you decommission resources that you no longer need or stop resources that are temporarily not needed?

Do you decommission resources that you no longer need or stop resources that are temporarily not needed?

Best practice

- Design your system to **gracefully handle instance termination** as you identify and decommission non-critical or unrequired instances or resources with low utilization
- Have a process in place to **identify and decommission** orphaned resources
- **Reconcile** decommissioned resources based on either system or process

Did you consider data-transfer charges when designing your architecture?

Did you consider data-transfer charges when designing your architecture?

Best practice

- Use the Amazon CloudFront CDN (content delivery network)
- Balance data transfer costs with high availability (HA) and reliability needs
- Architect to optimize data transfer
- Analyze if using AWS Direct Connect would save money and improve performance

Resources

- AWS Best Practices: https://d0.awsstatic.com/whitepapers/AWS_Cloud_Best_Practices.pdf
- AWS Well Architected Framework: https://d0.awsstatic.com/whitepapers/architecture/AWS_Well-Architected_Framework.pdf
- AWS SlideShare Channel: https://www.slideshare.net/AmazonWebServices/